

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO**

**ANDRÉ FABRICIO WOZNIACK
RICARDO HESPANHOL PAMPLONA**

**DETECÇÃO DE VULNERABILIDADES EM CÓDIGO-FONTE COM
MACHINE LEARNING**

PLANO DE PROJETO – 1º BIMESTRE

Orientador: Prof. Julio Cesar Nievola

Visto do Orientador

**CURITIBA
2026**

RESUMO

O crescente uso de ferramentas de Inteligência Artificial para geração automática de código, como GitHub Copilot, Claude Code e Codex, tem impulsionado a produtividade no desenvolvimento de software, mas também ampliado os desafios de segurança: levantamentos empíricos recentes indicam que o código assistido por IA pode apresentar taxas significativamente maiores de vulnerabilidades e recorrência de padrões inseguros quando comparado ao código tradicionalmente inspecionado por humanos. As ferramentas tradicionais de análise estática (SAST), amplamente utilizadas para identificar essas falhas, apresentam limitações relevantes, como elevadas taxas de falsos positivos e dificuldade em compreender o contexto semântico do código. Diante desse cenário, este trabalho tem como objetivo geral desenvolver um software capaz de analisar código-fonte antes de seu *merge* e integração à *branch* principal, identificando padrões associados a vulnerabilidades de segurança conhecidas por meio da combinação de análise estática e *Machine Learning*, auxiliados de forma complementar por *Large Language Models* (LLMs). A proposta adota uma abordagem modular e incremental: o Produto Mínimo Viável (MVP) estabelece um *pipeline* de análise estática e classificação supervisionada prioritariamente em linguagem Java, delimitado às categorias CWE-22, CWE-78, CWE-79 e CWE-89, enquanto representações avançadas em grafos (*Code Property Graphs*), suporte a C# e uso de LLMs configuram extensões avaliadas progressivamente. A validação será realizada por meio de testes de caixa branca, caixa preta e de integração, avaliando precisão, revocação e medida F1 em relação a linhas de base tradicionais. Espera-se, ao final do projeto, disponibilizar um protótipo funcional capaz de reduzir a quantidade de falsos positivos encaminhados ao desenvolvedor e de auxiliar na priorização de alertas de segurança em código-fonte.

Palavras-chave: Detecção de vulnerabilidades. Análise estática de código. *Machine Learning*. *Large Language Models*. Segurança de software.

SUMÁRIO

1	INTRODUÇÃO	4
1.1	MOTIVAÇÃO E RELEVÂNCIA	4
1.2	PERGUNTA DE PESQUISA	4
1.3	OBJETIVOS	5
1.3.1	OBJETIVO GERAL	5
1.3.2	OBJETIVOS ESPECÍFICOS	5
1.4	ESTRUTURA DO DOCUMENTO	6
2	DETALHAMENTO DO PROBLEMA	7
3	ESTADO DA ARTE	9
3.1	FERRAMENTAS DE ANÁLISE ESTÁTICA	9
3.2	ANÁLISE DINÂMICA E ABORDAGENS COMPLEMENTARES	11
3.3	MODELOS TRADICIONAIS DE <i>MACHINE LEARNING</i>	12
3.4	<i>DEEP LEARNING</i> PARA ANÁLISE DE CÓDIGO	13
3.5	MODELOS PRÉ-TREINADOS PARA CÓDIGO	14
3.6	<i>LARGE LANGUAGE MODELS</i> E SOLUÇÕES HÍBRIDAS	14
3.7	TRABALHOS RELACIONADOS: SEGURANÇA DE CÓDIGO GERADO POR IA	16
3.8	ANÁLISE CRÍTICA DAS SOLUÇÕES EXISTENTES	17
4	TRABALHO A SER DESENVOLVIDO	19
4.1	DESCRIÇÃO FUNCIONAL E DIAGRAMA EM BLOCOS	19
4.2	ESCOPO DA VERSÃO INICIAL	21
4.3	DETALHAMENTO DOS MÓDULOS FUNCIONAIS	22
4.3.1	MÓDULO DE INGESTÃO E NORMALIZAÇÃO DE CÓDIGO	22
4.3.2	MÓDULO DE ANÁLISE SINTÁTICA E ESTRUTURAL	22
4.3.3	MÓDULO DE REPRESENTAÇÃO INTERMEDIÁRIA E EXTRAÇÃO DE CARACTERÍSTICAS	23
4.3.4	MÓDULO DE PREPARAÇÃO E GESTÃO DE DADOS	23
4.3.5	MÓDULO DE DETECÇÃO DE VULNERABILIDADES	24
4.3.6	MÓDULO DE EXPLICAÇÃO E PRIORIZAÇÃO DE ALERTAS	24
4.3.7	MÓDULO DE RELATÓRIOS E INTERFACE	25
4.3.8	CONSIDERAÇÕES SOBRE <i>HARDWARE</i>	25

4.4	ESTRATÉGIA DE IMPLEMENTAÇÃO	25
5	TECNOLOGIAS QUE SERÃO UTILIZADAS	27
5.1	TECNOLOGIAS EM AVALIAÇÃO	27
5.1.1	LINGUAGENS DE ENTRADA E DE IMPLEMENTAÇÃO	27
5.1.2	ANÁLISE SINTÁTICA, ESTRUTURAL E ESTÁTICA	27
5.1.3	MODELOS DE CLASSIFICAÇÃO	28
5.1.4	RECURSOS DE <i>HARDWARE</i>	28
5.1.5	AMBIENTE DE DESENVOLVIMENTO E EXECUÇÃO	28
5.2	TECNOLOGIAS ALTERNATIVAS AVALIADAS	28
6	PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO	29
6.1	PROCEDIMENTOS DE TESTE	29
6.2	CRITÉRIOS DE ACEITE E DESEMPENHO	30
7	ANÁLISE DOS RISCOS	31
7.1	ANÁLISE DOS RISCOS	31
8	CRONOGRAMA DO PROJETO	33
9	CONCLUSÃO	35
	REFERÊNCIAS	38

1 INTRODUÇÃO

1.1 MOTIVAÇÃO E RELEVÂNCIA

Com a rápida difusão de assistentes de Inteligência Artificial para escrita de código, como GitHub Copilot, Claude Code e Codex, a taxa de produção de código acelerou consideravelmente, ampliando a necessidade de inspeção contínua de segurança. Embora a IA configure um contexto motivador central — haja vista estudos empíricos apontando que o código gerado tende a reproduzir vulnerabilidades recorrentes e padrões inseguros assimilados durante o treinamento dos modelos —, a demanda por verificação de segurança é universal e abrange qualquer código-fonte submetido a repositórios, seja ele redigido manualmente por humanos ou gerado com assistência de IA. Diante desse cenário, torna-se necessário adotar uma medida proativa para analisar código-fonte antes de seu *merge* e de sua integração definitiva à *branch* principal, reduzindo o risco de exploração de falhas em produção.

Embora existam ferramentas consolidadas de análise estática (SAST) que auxiliam na identificação de fraquezas de segurança, elas frequentemente sofrem com elevadas taxas de falsos positivos e exibem dificuldade para compreender o contexto semântico do código, como a intenção do desenvolvedor e o encadeamento de chamadas entre funções (TechScience; Yang *et al.*, 2025, 2025). Essa limitação torna o problema relevante do ponto de vista técnico e científico: não basta apontar possíveis violações com base em regras sintáticas rígidas; é imperativo enriquecer a análise com contexto semântico e estrutural para mitigar alarmes falsos e priorizar com precisão as vulnerabilidades críticas.

1.2 PERGUNTA DE PESQUISA

A partir do problema delineado e das limitações das abordagens estáticas convencionais, este trabalho orienta-se pela seguinte questão de pesquisa:

Em que medida a combinação de análise estática e classificação baseada em Aprendizado de Máquina reduz falsos positivos na detecção de CWE-22, CWE-78, CWE-79 e CWE-89 em código-fonte Java e C#?

1.3 OBJETIVOS

1.3.1 OBJETIVO GERAL

Desenvolver um software capaz de analisar código-fonte antes de seu *merge* e integração à *branch* principal, identificando padrões associados a vulnerabilidades de segurança conhecidas nas categorias CWE-22, CWE-78, CWE-79 e CWE-89. A solução fundamenta-se em uma arquitetura modular em que o Produto Mínimo Viável (MVP) provê um *pipeline* de análise estática e classificação por Aprendizado de Máquina (*Machine Learning*) com foco inicial em Java, enquanto representações avançadas em grafos (*Code Property Graphs*), suporte a C# e modelos de linguagem (*Large Language Models*) configuram componentes estendidos avaliados experimentalmente conforme a viabilidade técnica e o ganho mensurável.

1.3.2 OBJETIVOS ESPECÍFICOS

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- Realizar uma revisão bibliográfica sobre vulnerabilidades de segurança em código-fonte, ferramentas de análise estática, representações intermediárias em grafos e técnicas de *Machine Learning* e *Large Language Models* aplicadas à detecção de vulnerabilidades;
- Definir a arquitetura modular do sistema, delimitando o *pipeline* básico (MVP) e os módulos de extensão experimental;
- Selecionar as tecnologias e bases de dados abertas para o desenvolvimento do software, justificando suas escolhas por meio de critérios técnicos e provas de conceito;
- Implementar o software de forma incremental, garantindo a integração entre ingestão, extração de características e classificação;
- Validar a solução por meio de testes de caixa branca, caixa preta e de integração, mensurando precisão, revocação e medida F1 em comparação direta com ferramentas SAST de referência.

1.4 ESTRUTURA DO DOCUMENTO

Este documento está estruturado da seguinte forma: o Capítulo 2 detalha o problema a ser resolvido e a contextualização de riscos; o Capítulo 3 apresenta o estado da arte das soluções existentes e técnicas correlatas; o Capítulo 4 descreve o trabalho a ser desenvolvido, detalhando a arquitetura modular, o MVP e os módulos do sistema; o Capítulo 5 fundamenta as tecnologias selecionadas; o Capítulo 6 descreve os procedimentos de teste, métricas e critérios de aceite; o Capítulo 7 analisa os riscos técnicos e metodológicos do projeto; o Capítulo 8 apresenta o cronograma detalhado de execução; e o Capítulo 9 sintetiza as considerações finais deste Plano de Projeto.

2 DETALHAMENTO DO PROBLEMA

Historicamente, o código-fonte é produzido por desenvolvedores humanos, que escrevem manualmente cada linha, aplicando suas habilidades e conhecimentos para construir software funcional. Esse processo envolve a compreensão dos requisitos do sistema, a implementação de algoritmos e estruturas de dados e a realização de testes para garantir o funcionamento correto do software. Mesmo com desenvolvedores experientes, entretanto, erros podem ocorrer e resultar em vulnerabilidades de segurança.

Nos últimos anos, esse cenário tem sido transformado pelo advento de ferramentas de Inteligência Artificial voltadas ao desenvolvimento, como GitHub Copilot, Claude Code, Codex e Cursor, que utilizam modelos de linguagem avançados para auxiliar os desenvolvedores na escrita de código. Essas ferramentas podem gerar código automaticamente — desde pequenos trechos até projetos inteiros construídos por agentes a partir de descrições em linguagem natural. Embora aumentem a produtividade e reduzam o tempo de desenvolvimento, também introduzem novos desafios de segurança.

Estudos empíricos e levantamentos da indústria apontam que o código gerado ou assistido por IA pode apresentar um volume expressivo de falhas de segurança — com relatórios recentes indicando taxas de até 4,4 vezes mais vulnerabilidades em cenários específicos de desenvolvimento acelerado por assistentes inteligentes —, com determinados tipos de fraquezas aparecendo com frequência e *templates* inseguros se repetindo em diferentes repositórios, o que sugere padrões induzidos pelos próprios modelos (State of Cyber; Wu *et al.*, 2026, 2025).

Contudo, o desafio da segurança de código transcende a origem do artefato: seja o código escrito puramente por humanos ou produzido com auxílio de modelos gerativos, a necessidade de validação antes do *merge* e da integração definitiva à *branch* principal é imperativa. As ferramentas de análise estática tradicionalmente utilizadas para identificar esses problemas apresentam duas limitações centrais: elevada taxa de falsos positivos e dificuldade em compreender o contexto do código, como a intenção do desenvolvedor e o fluxo de dados entre funções

(Yang *et al.*; TechScience, 2025, 2025). Diante disso, há demanda por abordagens mais contextuais, baseadas na integração de análise estática a classificadores de *Machine Learning* e, de forma complementar, *Large Language Models*, capazes de detectar vulnerabilidades e priorizar alertas com maior precisão no fluxo contínuo de desenvolvimento (Wang *et al.*; Alqarni *et al.*, 2025b, 2026).

3 ESTADO DA ARTE

A detecção de vulnerabilidades em código-fonte é uma tarefa central em Engenharia de Software Segura e, historicamente, foi conduzida por revisões manuais e por ferramentas de análise estática. Nos últimos anos, com o avanço de técnicas de *Machine Learning* (ML) e, mais recentemente, dos *Large Language Models* (LLMs), surgiram novas abordagens capazes de aprender padrões diretamente de exemplos de código vulnerável, com o objetivo de reduzir falsos positivos e melhorar a sensibilidade ao contexto (Chakraborty, Krishna, Ding e Ray; Li *et al.*; Alqarni *et al.*, 2022, 2022, 2026). Este cenário se tornou ainda mais crítico com a popularização de assistentes de programação baseados em IA, como GitHub Copilot, Claude Code e Codex, cujo código produzido apresenta índices elevados de vulnerabilidades quando comparado ao código escrito por humanos (Pearce *et al.*; Veracode; Wu *et al.*, 2022, 2025, 2025).

3.1 FERRAMENTAS DE ANÁLISE ESTÁTICA

A análise estática de software, também conhecida como *Static Application Security Testing* (SAST), consiste na inspeção do código-fonte, do *bytecode* ou do binário sem a sua execução, com o objetivo de identificar defeitos, violações de estilo e, em particular, vulnerabilidades de segurança (TechScience, 2025). Diferentemente das abordagens dinâmicas discutidas na Seção 3.2, a análise estática opera sobre uma representação intermediária do código, sem executá-lo, e é aplicada tipicamente durante o desenvolvimento ou nos processos de integração contínua e entrega contínua (*Continuous Integration/Continuous Delivery* — CI/CD).

Internamente, um analisador estático constrói representações intermediárias do programa, como *Abstract Syntax Trees* (ASTs), *Control Flow Graphs* (CFGs) e *Data Flow Graphs* (DFGs). Sobre essas estruturas são executadas técnicas como **casamento de padrões sintáticos**, **análise de fluxo de dados**, **análise interprocedural** e, notadamente, a **análise de taint** (*taint analysis*), que rastreia se dados provenientes de fontes não confiáveis, chamadas *sources*, por exemplo, parâmetros de requisições HTTP ou leitura de arquivos, atingem operações sensíveis, chamadas *sinks*, como consultas SQL, chamadas de sistema operacional ou renderização de

HTML, sem passar por uma sanitização adequada (Yamaguchi, Golde, Arp e Rieck; Safeguard.sh, 2014, 2024). Uma representação particularmente relevante é o *Code Property Graph* (CPG), proposto por Yamaguchi et al. (Yamaguchi, Golde, Arp e Rieck, 2014), que combina AST, CFG e grafo de dependências de programa em uma única estrutura consultável por uma linguagem específica de domínio, servindo de base para ferramentas como Joern e para produtos comerciais mais recentes.

Do ponto de vista prático, ferramentas amplamente adotadas na indústria incluem SonarQube, Semgrep, CodeQL, Checkmarx, Fortify e Snyk Code. Essas ferramentas identificam padrões relacionados a categorias de vulnerabilidades da *Common Weakness Enumeration* (CWE), como uso de funções inseguras (`strcpy`, `gets`, `eval`), validação inadequada de entradas, injeção de comandos, injeção SQL, *buffer overflow*, *Cross-Site Scripting* (XSS) e problemas de gerenciamento de memória. Muitas delas permitem, ainda, a definição de regras personalizadas, adequadas às convenções e aos requisitos de segurança de cada organização.

A principal vantagem dessa família de ferramentas é a **explicabilidade**: cada alerta é acompanhado do trecho de código associado, da regra violada e, com frequência, de orientações para a correção. Isso as torna adequadas para revisão de código e para integração em *pipelines* de CI/CD, com custo computacional relativamente baixo e sem dependência de dados de treinamento.

Entretanto, essas soluções apresentam limitações bem documentadas. Em primeiro lugar, sua capacidade de detecção depende de regras predefinidas, o que reduz a eficácia diante de padrões novos ou não modelados. Em segundo lugar, elas produzem elevadas taxas de **falsos positivos**, alertas que apontam trechos aparentemente vulneráveis, mas que não representam risco real no contexto da aplicação, o que pode saturar equipes de desenvolvimento e reduzir a confiança nos relatórios (TechScience; Yang et al., 2025, 2025). Estudos recentes mostram que frameworks que combinam análise estática com LLMs, como o ZeroFalse (Yang et al., 2025) e propostas correlatas (Wen et al., 2025), chegam a eliminar entre 94% e 98% dos falsos alarmes em contextos industriais, o que evidencia tanto a magnitude do problema quanto o potencial das abordagens híbridas. Essas limitações motivaram a investigação de técnicas de ML capazes de aprender padrões a partir de exemplos de código vulnerável.

3.2 ANÁLISE DINÂMICA E ABORDAGENS COMPLEMENTARES

Embora este trabalho concentre-se em análise estática, é importante situá-la em relação às demais famílias de *Application Security Testing* (AST), que se distinguem principalmente pelo momento do ciclo de desenvolvimento em que atuam e pelo acesso ao código-fonte.

O *Dynamic Application Security Testing* (DAST) analisa a aplicação em execução, tipicamente em ambiente de *staging* ou homologação, sem acesso ao código-fonte. Ferramentas como OWASP ZAP, Burp Suite e Nuclei operam como um agente externo que envia requisições maliciosas, como cargas de *SQL Injection*, *XSS*, *path traversal*, entre outras e observa as respostas da aplicação para inferir a presença de vulnerabilidades (OWASP Foundation; Ranasinghe *et al.*, 2026, 2025). Por atuar sobre o sistema em funcionamento, o DAST identifica falhas que dificilmente aparecem em análise estática, como problemas de configuração de servidor, falhas de autenticação e autorização, *Security Misconfiguration* e *Broken Access Control*, categorias em que o DAST supera o SAST em avaliações empíricas recentes (Ranasinghe *et al.*, 2025). Em contrapartida, o DAST depende de uma aplicação executável e razoavelmente instrumentada, não aponta a localização exata da vulnerabilidade no código-fonte e tende a apresentar cobertura inferior à da análise estática, já que só identifica falhas nos caminhos efetivamente exercitados durante o *scan*.

Entre SAST e DAST, o *Interactive Application Security Testing* (IAST) atua como abordagem híbrida: instrumenta a aplicação em tempo de execução por meio de agentes ou *sensors*, de forma que as vulnerabilidades sejam observadas durante a execução, mas com acesso ao código e ao fluxo de dados internos. Essa combinação tende a produzir menos falsos positivos, ao custo de exigir instrumentação e cobertura adequada de testes funcionais (Elder, Rahman, Fu e Williams; OWASP Foundation, 2023, 2026). Já o *Runtime Application Self-Protection* (RASP) atua em produção, bloqueando ativamente tentativas de exploração e é complementar, não substituto às demais abordagens.

Na prática, essas famílias são adotadas de forma **complementar** ao longo do ciclo de vida do *software*: SAST durante o desenvolvimento e nos *pipelines* de CI/CD; DAST em fases de homologação e pré-produção; IAST em testes automatizados; e RASP em produção (OWASP Foundation, 2026). O presente trabalho posiciona-se

na etapa inicial desse ciclo, atuando sobre o código-fonte antes de sua integração ao sistema, recorte particularmente relevante para o cenário de código gerado por LLMs, em que muitas vezes ainda não existe uma aplicação executável para submeter a técnicas dinâmicas. Por essa razão, as próximas seções concentram-se em abordagens baseadas em análise estática, aprendizado de máquina e modelos de linguagem aplicados ao código-fonte.

3.3 MODELOS TRADICIONAIS DE *MACHINE LEARNING*

As primeiras pesquisas que aplicaram ML à detecção de vulnerabilidades utilizaram **características extraídas manualmente** do código-fonte (*hand-crafted features*). Entre os atributos usualmente extraídos estão métricas de software como quantidade de linhas de código e complexidade ciclomática, frequência de chamadas a funções consideradas inseguras, contagem de operadores e operandos, uso de APIs específicas, tokens léxicos e propriedades derivadas da AST.

A partir dessas características, algoritmos clássicos como *Random Forest*, *Support Vector Machine* (SVM), árvores de decisão, regressão logística e *Naive Bayes* podem ser treinados para classificar funções, arquivos ou *commits* como vulneráveis ou não vulneráveis. Essas abordagens possuem, em geral, menor custo computacional quando comparadas a redes neurais profundas, são mais simples de implementar e apresentam maior interpretabilidade, uma vez que a contribuição de cada atributo pode ser inspecionada.

Entretanto, a eficácia desses modelos está diretamente ligada à qualidade dos atributos selecionados. Como vulnerabilidades podem depender do fluxo de dados, das relações entre variáveis, das chamadas de funções e do contexto de execução, características superficiais nem sempre representam adequadamente o comportamento do programa. Assim, modelos tradicionais tendem a apresentar dificuldade para detectar falhas complexas, especialmente quando o padrão vulnerável não está restrito a uma única linha ou função (Chakraborty, Krishna, Ding e Ray, 2022).

3.4 DEEP LEARNING PARA ANÁLISE DE CÓDIGO

Com o avanço das técnicas de *Deep Learning*, surgiram modelos capazes de aprender representações diretamente do código-fonte, reduzindo a necessidade de definir atributos manualmente. Entre as arquiteturas aplicadas ao problema estão redes neurais convolucionais (*Convolutional Neural Networks* — CNNs), redes neurais recorrentes (*Recurrent Neural Networks* — RNNs) como LSTM e GRU, *Transformers* e *Graph Neural Networks* (GNNs).

Uma solução de referência é o **Devign**, proposto por Zhou et al. (Zhou et al., 2019). O método representa o código por meio de grafos compostos, integrando AST, CFG, DFG e grafo de dependência natural, e aplica uma *Gated Graph Neural Network* para identificar funções vulneráveis em C/C++. Sua proposta considera não apenas os tokens presentes no código, mas também relações estruturais e semânticas entre instruções, o que é relevante porque uma vulnerabilidade pode decorrer da interação entre diferentes partes de uma função, e não apenas de uma instrução isolada.

Abordagens baseadas em grafos, em geral, exploram estruturas como AST, CFG, DFG e CPG. Essas estruturas permitem representar, respectivamente, a sintaxe do programa, os possíveis caminhos de execução, o fluxo de informações entre variáveis e uma visão unificada dessas relações. Modelos baseados em grafos são especialmente adequados para vulnerabilidades relacionadas à manipulação de memória, validação de entradas e uso incorreto de variáveis (Chakraborty, Krishna, Ding e Ray; Yamaguchi, Golde, Arp e Rieck, 2022, 2014).

Apesar dos resultados positivos, essas abordagens apresentam desafios recorrentes. A construção dos grafos exige ferramentas adicionais de análise de código; o custo computacional cresce com o tamanho e a complexidade do projeto; e diversos estudos avaliam os modelos apenas no nível de função, enquanto vulnerabilidades reais podem depender da interação entre arquivos, classes, módulos ou chamadas interprocedurais (Chakraborty, Krishna, Ding e Ray; Ding et al., 2022, 2024).

3.5 MODELOS PRÉ-TREINADOS PARA CÓDIGO

Uma outra abordagem recente utiliza modelos de linguagem pré-treinados especificamente em código-fonte. Esses modelos aprendem padrões a partir de grandes coleções de programas e podem ser adaptados por *fine-tuning* para tarefas como classificação de código vulnerável, identificação de categorias CWE e sugestão de correções.

O **CodeBERT**, proposto por Feng et al. (Feng *et al.*, 2020), é um modelo baseado em *Transformers* que aprende representações conjuntas de código-fonte e linguagem natural, treinado sobre pares comentário-código em seis linguagens. Após o *fine-tuning* com exemplos de funções vulneráveis e não vulneráveis, o modelo pode ser utilizado em tarefas de classificação.

O **GraphCodeBERT**, proposto por Guo et al. (Guo *et al.*, 2021), amplia essa ideia ao incorporar informações de fluxo de dados durante o pré-treinamento. Dessa forma, o modelo busca preservar relações relevantes entre variáveis e operações, que não são adequadamente representadas quando o código é tratado somente como uma sequência de tokens.

Essas soluções são promissoras porque conseguem lidar com dependências contextuais de longo alcance. Por exemplo, uma variável pode receber um valor em determinada parte do código e ser utilizada de forma insegura várias linhas depois; um modelo baseado em *Transformers* pode aprender relações desse tipo com maior eficiência do que métodos baseados apenas em frequência de tokens.

No entanto, modelos baseados exclusivamente em sequências ainda podem perder informações estruturais relevantes. Por esse motivo, pesquisas recentes combinam *Transformers* com representações em grafo, a fim de unir a compreensão contextual dos modelos de linguagem à representação estrutural oferecida pelos grafos de código (Guo *et al.*; Chakraborty, Krishna, Ding e Ray; Zhou *et al.*, 2021, 2022, 2026).

3.6 LARGE LANGUAGE MODELS E SOLUÇÕES HÍBRIDAS

Atualmente, os *Large Language Models* voltados à programação como Code Llama, StarCoder, DeepSeek-Coder, GPT-4 e Claude, vêm sendo explorados para detectar vulnerabilidades, explicar os riscos encontrados e sugerir possíveis correções

(Alqarni *et al.*, 2026). Diferentemente de classificadores tradicionais, esses modelos podem analisar um trecho de código e gerar descrições em linguagem natural, indicando uma possível vulnerabilidade, sua categoria CWE e uma recomendação de mitigação.

Uma tendência clara é a integração de LLMs com **Code Property Graphs**. Nessa combinação, o grafo fornece informações estruturais precisas, fluxo de dados, fluxo de controle e relações interprocedurais, enquanto o LLM contribui com a interpretação contextual do código. O **LLMxCPG** (Kouliaridis *et al.*, 2025) explora essa estratégia para detecção sensível a contexto, e o **SV-GEN** (Zhou *et al.*, 2026) combina semântica de variáveis extraída por LLM com *Graph Transformers* para melhorar a detecção em cenários com poucos rótulos.

Outra frente relevante é o uso de LLMs como **filtros de falsos positivos** sobre alertas de análise estática tradicional. O **ZeroFalse** (Yang *et al.*, 2025) trata as saídas dos analisadores como contratos estruturados, enriquece-os com traços sensíveis ao fluxo e com conhecimento específico da CWE em questão, e utiliza um LLM como árbitro final. Abordagens semelhantes, como LLM4PFA (Wen *et al.*, 2025) e AdaTaint (Wang *et al.*, 2025a), aplicam raciocínio iterativo sobre caminhos de execução e adaptação de *sources/sinks*, alcançando reduções expressivas de falsos alarmes preservando a cobertura da análise original.

Apesar de seu potencial, os LLMs possuem limitações importantes. Podem gerar falsos positivos, deixar de identificar vulnerabilidades reais ou fornecer justificativas aparentemente plausíveis, mas incorretas (*hallucinations*). Também existem questões relacionadas ao custo computacional, à **privacidade do código-fonte** enviado a serviços externos e à dificuldade de reproduzir resultados quando modelos proprietários são utilizados. Além disso, a capacidade de generalização permanece um desafio: modelos treinados em um conjunto de dados específico podem apresentar desempenho significativamente inferior quando aplicados a projetos, linguagens ou distribuições de dados diferentes (Ding *et al.*; Bhusal *et al.*, 2024, 2025a).

3.7 TRABALHOS RELACIONADOS: SEGURANÇA DE CÓDIGO GERADO POR IA

Uma vez que o presente trabalho tem como foco a análise de código-fonte antes de sua integração a um sistema, em particular considerando o crescente uso de assistentes de IA na produção de código, esta seção apresenta os trabalhos mais próximos, especificamente voltados à segurança de código gerado por LLMs.

O estudo pioneiro nessa direção foi conduzido por Pearce et al. (Pearce *et al.*, 2022), que avaliaram empiricamente a segurança do GitHub Copilot em 1.689 programas gerados a partir de cenários relacionados às 25 principais CWEs; aproximadamente 40% dos programas continham vulnerabilidades. Trabalhos posteriores estenderam essa análise para outros LLMs e cenários. Perry et al. (Perry, Srivastava, Kumar e Boneh, 2023) observaram que desenvolvedores assistidos por LLMs tendem a produzir código menos seguro, ao mesmo tempo em que confiam mais em sua segurança do que deveriam. O relatório da Veracode (Veracode, 2025) analisou o código produzido por mais de 100 LLMs em 80 tarefas de programação reais e reportou que aproximadamente 45% do código gerado continha vulnerabilidades, com padrões inseguros recorrentes entre diferentes modelos e linguagens.

Já Wu et al. (Wu *et al.*, 2025) propuseram uma medição em escala de código gerado por IA em repositórios públicos, quantificando os riscos e avaliando estratégias de mitigação. Uma constatação recorrente é que determinados *templates* inseguros se repetem em diferentes projetos, sugerindo padrões induzidos pelos próprios modelos (Wu *et al.*; State of Cyber, 2025, 2026). Trabalhos como o de Liu et al. (Liu *et al.*, 2025) exploram, ainda, o uso de dicas de vulnerabilidade e *feedback* de granularidade fina para induzir LLMs a gerar código mais seguro, indicando que o problema também pode ser mitigado no lado da geração, e não apenas no da detecção.

Do ponto de vista de **datasets e benchmarks**, a literatura consolidou algumas bases de referência para treinar e comparar detectores de vulnerabilidades. O **BigVul** (Fan, Li, Wang e Nguyen, 2020) reúne funções vulneráveis em C/C++ extraídas de projetos abertos com base em correções associadas a CVEs. O **DiverseVul** (Chen *et al.*, 2023) amplia essa cobertura e mostra que rótulos derivados automaticamente apresentam ruído significativo, o que motivou a construção de bases mais cuidadosas. O **PrimeVul** (Ding *et al.*, 2024) introduz técnicas de rotulagem que aumentam a precisão dos rótulos e uma divisão temporal que evita vazamento entre treino e teste;

nele, mesmo modelos grandes ajustados para código exibem quedas expressivas de desempenho em relação a benchmarks anteriores, revelando que resultados tradicionalmente reportados podem estar superestimados. Benchmarks mais recentes, como o SecVulEval (Hasan *et al.*, 2025) e o BenchVul (Bhusal *et al.*, 2025b), focam em avaliação por CWE e em generalização *out-of-distribution*, aproximando as métricas de cenários realistas.

Uma síntese comparativa das principais famílias de abordagens é apresentada na Tabela 1, considerando critérios relevantes para o presente trabalho.

3.8 ANÁLISE CRÍTICA DAS SOLUÇÕES EXISTENTES

Tabela 1 – Comparativo qualitativo das abordagens de detecção de vulnerabilidades em código-fonte.

Abordagem		Explicabili- dade	Custo compu- tacional	Falsos positivos	Generalização
Análise (SAST)	estática	Alta	Baixo	Altos	Baixa (regras fixas)
Análise (DAST)	dinâmica	Média	Médio	Baixos	Cobertura limi- tada
ML tradicional		Média	Baixo/Médio	Médios	Média
Deep Learning (CNN, RNN, GNN)		Baixa	Médio/Alto	Médios	Média
Pré-treinados (CodeBERT, GraphCodeBERT)		Baixa/Média	Alto	Médios	Média/Alta
LLMs + CPG (híbridos)		Média/Alta	Alto	Baixos	Alta

A análise das soluções existentes mostra que não há uma abordagem única capaz de resolver completamente o problema de detecção de vulnerabilidades em código-fonte. As ferramentas de análise estática permanecem relevantes por sua maturidade, explicabilidade, rapidez e facilidade de integração aos processos de desenvolvimento. Entretanto, sua dependência de regras predefinidas limita a identificação de padrões novos ou complexos, e as elevadas taxas de falsos positivos comprometem sua adoção efetiva em larga escala (TechScience; Yang *et al.*, 2025, 2025).

Os métodos tradicionais de ML representam uma alternativa às regras fixas, pois aprendem padrões a partir de dados. Contudo, sua eficácia depende da seleção de

atributos e apresentam limitações para representar o contexto semântico e as relações de fluxo de dados presentes no programa.

As abordagens baseadas em *Deep Learning*, *Transformers* e grafos de código apresentam maior potencial para modelar características sintáticas, semânticas e estruturais. Em particular, soluções híbridas que combinam LLMs a grafos de código ou a analisadores estáticos são promissoras, pois integram a capacidade de compreensão contextual dos modelos de linguagem à representação explícita do fluxo de controle e do fluxo de dados (Zhou *et al.*; Kouliaridis *et al.*; Yang *et al.*, 2026, 2025, 2025).

Uma abordagem, entretanto, não deve ser considerada superior apenas por apresentar maior acurácia em um único conjunto de dados. Estudos como os de Ding *et al.* (Ding *et al.*, 2024) evidenciam que resultados elevados em benchmarks tradicionais podem cair drasticamente em condições realistas, com divisão temporal e rótulos revisados. A comparação entre abordagens deve, portanto, considerar métricas como precisão, *recall*, *F1-score*, taxa de falsos positivos, custo computacional, tempo de inferência, explicabilidade e generalização para projetos não utilizados no treinamento. Também é necessário considerar a qualidade dos rótulos e o nível de análise adotado, pois muitos datasets avaliam apenas funções isoladas, enquanto vulnerabilidades reais podem depender de contexto interprocedural.

Especificamente para o cenário de código gerado por IA, os trabalhos relacionados discutidos na Seção 3.7 concentram-se predominantemente em **medir** a insegurança do código produzido ou em induzir os LLMs a gerar código mais seguro. Observa-se, assim, uma oportunidade para soluções que atuem na etapa **intermediária**, ou seja, na análise do código gerado por IA antes de sua integração ao sistema, combinando a rastreabilidade das ferramentas estáticas com a compreensão contextual dos LLMs para reduzir falsos positivos e priorizar alertas críticos.

4 TRABALHO A SER DESENVOLVIDO

4.1 DESCRIÇÃO FUNCIONAL E DIAGRAMA EM BLOCOS

Este projeto propõe o desenvolvimento de uma ferramenta de apoio à segurança de software capaz de analisar código-fonte e identificar possíveis vulnerabilidades antes de seu *merge* e de sua integração definitiva à *branch* principal. A solução será orientada à detecção de padrões associados a fraquezas de segurança conhecidas, utilizando prioritariamente técnicas de análise estática e *Machine Learning*, auxiliadas de forma complementar por *Large Language Models*. A ferramenta buscará complementar os analisadores estáticos tradicionais e reduzir a quantidade de falsos positivos encaminhados ao desenvolvedor.

Para assegurar a viabilidade técnica e a aderência aos prazos acadêmicos, a arquitetura do sistema é dividida em dois níveis de maturidade:

- a) **Arquitetura-Base (Produto Mínimo Viável - MVP)**: constitui o *pipeline* essencial e executável da pesquisa. Concentra-se nas linguagens Java e C#, na ingestão de alterações em nível de método ou função (a unidade elementar de análise do classificador), na extração de estruturas sintáticas básicas (AST) e regras de análise estática de referência (SAST), e no treinamento de um classificador supervisionado de *Machine Learning* para categorizar as quatro CWEs do escopo;
- b) **Arquitetura Estendida (Prova de Conceito e Módulos Opcionais)**: engloba a investigação de *Code Property Graphs* (CPGs) e o emprego de LLMs para sumarização explicativa dos alertas. Esses componentes serão incorporados e consolidados apenas se as provas de conceito demonstrarem ganho mensurável de eficácia frente ao custo computacional.

O sistema receberá como entrada os arquivos modificados em um *commit* ou *pull request*. A proposta inicial é normalizar o conteúdo e processá-lo, delimitando como unidade de análise os métodos ou blocos de código impactados pelas alterações. Em seguida, serão extraídas informações sintáticas, estruturais e de fluxo. Essas

informações serão convertidas para uma representação estruturada, utilizada como entrada pelo módulo de detecção e classificação de vulnerabilidades.

O módulo de detecção classificará as funções analisadas de acordo com a possível presença de padrões associados às CWEs definidas no escopo do projeto (CWE-22, CWE-78, CWE-79 e CWE-89). Como resultado, a ferramenta deverá apresentar a categoria da fraqueza identificada, a localização no código (arquivo e linha), o nível de confiança da classificação e, quando aplicável, uma explicação contextualizada ou recomendação que auxilie a revisão pelo desenvolvedor.

A Figura 1 apresenta uma visão geral da arquitetura proposta e do fluxo de processamento da solução.

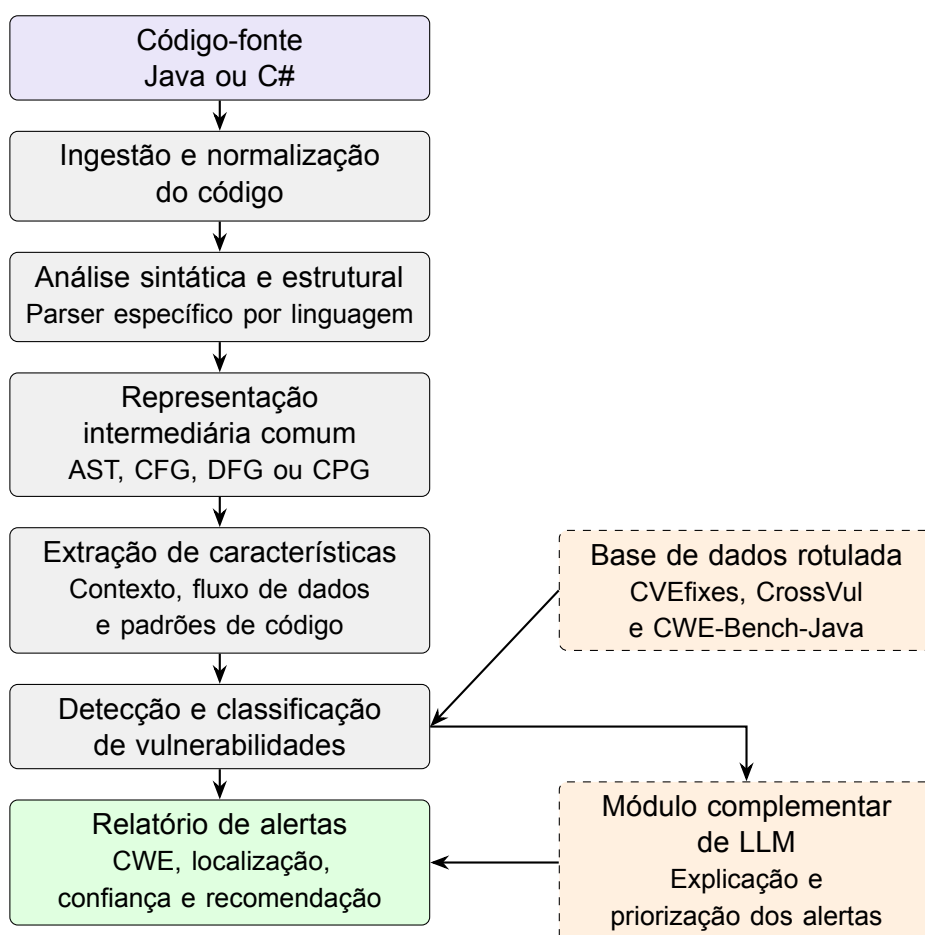


Figura 1 – Visão geral da arquitetura proposta para a ferramenta de detecção de vulnerabilidades.

Fonte: Elaborado pelo autor.

4.2 ESCOPO DA VERSÃO INICIAL

A versão inicial da solução será desenvolvida para analisar código-fonte escrito nas linguagens Java e C#. O sistema terá como foco a identificação de padrões associados às CWEs 22 (*Path Traversal*), 78 (*OS Command Injection*), 79 (*Cross-Site Scripting*) e 89 (*SQL Injection*).

A seleção dessas categorias busca equilibrar relevância prática, possibilidade de análise por fluxo de dados, disponibilidade de exemplos rotulados em conjuntos de dados públicos e viabilidade de implementação dentro do cronograma acadêmico. A Tabela 2 apresenta as fraquezas inicialmente contempladas pela ferramenta.

Tabela 2 – CWEs previstas para a versão inicial da ferramenta.

CWE	Fraqueza	Descrição
CWE-22	<i>Path Traversal</i>	Acesso indevido a arquivos ou diretórios decorrente da manipulação inadequada de caminhos fornecidos por usuários.
CWE-78	<i>OS Command Injection</i>	Execução não autorizada de comandos do sistema operacional a partir de entradas não confiáveis.
CWE-79	<i>Cross-Site Scripting</i>	Geração de conteúdo web sem a neutralização adequada de dados controlados por usuários.
CWE-89	<i>SQL Injection</i>	Construção insegura de consultas a bancos de dados com dados externos não validados ou não parametrizados.

Fonte: Elaborado pelo autor.

Em evoluções posteriores, pretende-se ampliar a cobertura da ferramenta para as CWEs 862 (*Missing Authorization*), 287 (*Improper Authentication*), 918 (*Server-Side Request Forgery – SSRF*) e 94 (*Code Injection*). Essa expansão dependerá da disponibilidade de dados adequados, da validação do fluxo inicial de análise e da conclusão das atividades previstas para a versão mínima viável.

A arquitetura será organizada de forma modular, separando os componentes específicos de cada linguagem de uma camada comum de representação, classificação e geração de relatórios. Assim, embora o produto mínimo viável suporte inicialmente Java e C#, a solução será preparada para a incorporação futura de outras linguagens, sem a necessidade de reconstrução completa do sistema.

Os conjuntos CVEfixes e CrossVul serão investigados como fontes de exemplos vulneráveis e corrigidos para as linguagens-alvo. O CWE-Bench-Java será utilizado

como referência complementar de avaliação para as categorias contempladas em sua composição.

4.3 DETALHAMENTO DOS MÓDULOS FUNCIONAIS

A solução será decomposta em módulos funcionais para favorecer a organização, a implementação incremental, os testes individuais e a evolução futura da ferramenta.

4.3.1 MÓDULO DE INGESTÃO E NORMALIZAÇÃO DE CÓDIGO

Este módulo será responsável por receber arquivos, diretórios ou trechos de código-fonte fornecidos pelo usuário. Inicialmente, verificará se a entrada corresponde a código Java ou C# e executará procedimentos de normalização, como a padronização da codificação, a remoção de elementos irrelevantes para a análise e a divisão do conteúdo em unidades analisáveis.

Após a normalização, os artefatos serão encaminhados aos componentes responsáveis pela análise sintática e estrutural e pela construção de uma representação intermediária do código. Pretende-se investigar o uso de *Code Property Graphs* (CPGs), que permitem integrar informações provenientes de árvores sintáticas, fluxos de controle e fluxos de dados em uma representação baseada em grafos.

Entre as alternativas tecnológicas, serão avaliados o ecossistema CPG/Codyze, o Joern e bibliotecas específicas de análise sintática. A adoção de um CPG dependerá do benefício obtido para as CWEs selecionadas, do suporte às linguagens Java e C# e do custo de integração. Caso essa representação não se mostre adequada, serão mantidas estruturas mais simples, como ASTs, CFGs e DFGs. A tecnologia de implementação e o formato de intercâmbio entre os módulos serão definidos durante o desenvolvimento, priorizando interoperabilidade, extensibilidade e representação estruturada dos dados.

4.3.2 MÓDULO DE ANÁLISE SINTÁTICA E ESTRUTURAL

O módulo de análise sintática interpretará o código de acordo com as regras das linguagens Java e C#. Para isso, serão utilizados parsers ou bibliotecas específicas

capazes de gerar estruturas como árvores sintáticas abstratas (*Abstract Syntax Trees* – ASTs).

Também serão investigadas informações relacionadas ao fluxo de controle e ao fluxo de dados. Essas informações são necessárias para identificar situações em que dados externos percorrem o programa e alcançam operações potencialmente sensíveis, tais como execução de comandos, acesso a arquivos, geração de conteúdo web ou consultas a bancos de dados.

Como fonte complementar, o módulo poderá orquestrar ferramentas consagradas de análise estática (SAST), como SonarQube, Semgrep e *linters* específicos de cada linguagem. (Ferramentas de análise de vulnerabilidades em dependências externas/SCA, como o Dependabot, não fazem parte da análise direta de código-fonte aqui proposta e ficam reservadas para trabalhos futuros). Os alertas produzidos pelas ferramentas SAST serão normalizados, preservando arquivo, linha, categoria e severidade, para que possam ser combinados com as características extraídas diretamente do código. Quando aplicável, essas ferramentas serão executadas em contêineres Docker para favorecer a reprodutibilidade do ambiente.

4.3.3 MÓDULO DE REPRESENTAÇÃO INTERMEDIÁRIA E EXTRAÇÃO DE CARACTERÍSTICAS

As estruturas produzidas pelos parsers serão convertidas para uma representação intermediária comum. Essa camada busca reduzir a dependência direta de detalhes sintáticos de Java e C#, permitindo que os módulos de extração de características e classificação operem sobre informações mais uniformes.

A partir dessa representação, serão extraídas características léxicas, sintáticas e estruturais, tais como chamadas de métodos, operações sensíveis, relações entre variáveis, caminhos de fluxo de dados e contexto das instruções. Essas características serão utilizadas como entrada pelo modelo de detecção de vulnerabilidades.

4.3.4 MÓDULO DE PREPARAÇÃO E GESTÃO DE DADOS

Este módulo será responsável por obter, filtrar, limpar e organizar os dados empregados no treinamento e na avaliação dos modelos. Serão considerados

exemplos de código vulnerável e corrigido, acompanhados de informações como CVE, CWE, projeto de origem e alteração de correção.

Para garantir a cobertura empírica das linguagens e fraquezas abordadas, o projeto utilizará bases públicas consolidadas: o *CVEfixes* e o *CrossVul*, que reúnem históricos de correções de segurança em nível de *commit* e método com ampla representatividade para Java e C# nas categorias de injeção e travessia de diretório (CWE-22, CWE-78, CWE-79 e CWE-89), além do *CWE-Bench-Java*, que fornece instâncias curadas e verificadas para a validação rigorosa da linguagem prioritária do MVP.

Para reduzir ameaças à validade dos resultados, os dados serão separados em conjuntos de treinamento, validação e teste. A separação será rigorosamente realizada por projeto ou repositório, evitando que versões muito semelhantes de um mesmo código sejam utilizadas simultaneamente no treinamento e na avaliação (Ding *et al.*, 2024).

4.3.5 MÓDULO DE DETECÇÃO DE VULNERABILIDADES

O módulo de detecção receberá as características extraídas do código e utilizará modelos de *Machine Learning* para estimar a ocorrência de vulnerabilidades relacionadas às CWEs definidas no escopo do projeto. Inicialmente, serão adotados modelos de referência mais simples, capazes de estabelecer uma linha de base para comparação.

Posteriormente, poderão ser avaliadas abordagens baseadas em modelos pré-treinados para código ou em representações de grafos. A seleção do modelo considerará não apenas o desempenho nas métricas de classificação, mas também o custo computacional, a reprodutibilidade e a possibilidade de explicar os alertas gerados.

4.3.6 MÓDULO DE EXPLICAÇÃO E PRIORIZAÇÃO DE ALERTAS

Após a identificação de uma possível vulnerabilidade, este módulo organizará os resultados de modo a facilitar a revisão pelo desenvolvedor. Cada alerta deverá indicar, quando disponível, a CWE associada, o arquivo, o trecho de código, o nível de confiança e a razão principal para a sinalização.

Um *Large Language Model* poderá ser utilizado de forma complementar para gerar explicações em linguagem natural e sugerir ações de mitigação. Esse componente não será a única fonte de decisão do sistema: a classificação principal permanecerá baseada no pipeline de análise e no modelo treinado. Essa decisão reduz o risco de alertas ou explicações sem evidência suficiente no código analisado.

4.3.7 MÓDULO DE RELATÓRIOS E INTERFACE

O módulo de relatórios apresentará os resultados da análise em formato compreensível para desenvolvedores. Inicialmente, a saída poderá ser disponibilizada em terminal, arquivo estruturado ou interface simples, contendo o resumo da análise e os alertas encontrados.

A ferramenta deverá informar as CWEs identificadas, a localização aproximada dos problemas, a confiança da classificação e recomendações básicas de tratamento. Em versões futuras, esse módulo poderá ser integrado a ambientes de desenvolvimento, repositórios de código ou pipelines de integração contínua.

A linguagem de implementação será definida após as provas de conceito descritas no Capítulo 5. A versão inicial deverá gerar relatórios em formato estruturado e oferecer uma interface de linha de comando, permitindo tanto a execução manual quanto a integração a um *pipeline* de integração contínua.

4.3.8 CONSIDERAÇÕES SOBRE *HARDWARE*

O projeto não prevê o desenvolvimento de módulos de *hardware*. A solução será executada em computadores de propósito geral. Os computadores pessoais dos integrantes serão utilizados no desenvolvimento e nos testes do *pipeline*; infraestrutura local ou em nuvem poderá ser empregada quando o treinamento dos modelos exigir maior capacidade de processamento, especialmente na presença de GPU.

4.4 ESTRATÉGIA DE IMPLEMENTAÇÃO

O desenvolvimento será realizado de forma incremental. Inicialmente, será implementado um protótipo capaz de receber código Java e C#, normalizar a entrada e produzir uma representação estruturada. Em seguida, serão integrados

os componentes de extração de características e um modelo de classificação de referência.

Após a validação do fluxo básico, serão incorporados mecanismos de priorização de alertas, geração de relatórios e, se tecnicamente viável, um módulo complementar baseado em *Large Language Models*. Essa estratégia busca garantir a entrega de um produto mínimo viável antes da inclusão de funcionalidades mais complexas.

5 TECNOLOGIAS QUE SERÃO UTILIZADAS

5.1 TECNOLOGIAS EM AVALIAÇÃO

As tecnologias de implementação ainda não foram definidas de forma definitiva. Como o projeto combina análise estática, representações estruturadas de código e modelos de *Machine Learning*, a escolha será realizada por meio de provas de conceito, considerando o suporte a Java e C#, o custo computacional, a integração entre os módulos e a reprodutibilidade dos resultados.

5.1.1 LINGUAGENS DE ENTRADA E DE IMPLEMENTAÇÃO

Java e C# foram definidas como as linguagens de entrada da versão inicial da ferramenta por serem utilizadas em ambientes corporativos. A linguagem de implementação do *pipeline*, entretanto, será selecionada durante o desenvolvimento. Python é uma alternativa devido ao seu ecossistema de ML; Java ou Kotlin podem facilitar a integração com a JVM; e C#.NET permite o uso de APIs de compilação e análise de projetos dessa linguagem.

A decisão considerará a experiência da equipe e as bibliotecas disponíveis. Formatos estruturados, como JSON, poderão ser utilizados para reduzir o acoplamento entre os módulos.

5.1.2 ANÁLISE SINTÁTICA, ESTRUTURAL E ESTÁTICA

A representação inicial poderá utilizar árvores sintáticas abstratas, complementadas por informações de fluxo de controle e de dados. O uso de *Code Property Graphs* será avaliado por meio de uma prova de conceito, pois os CPGs reúnem essas representações em um único grafo (Yamaguchi, Golde, Arp e Rieck, 2014).

CPG/Codyze e Joern serão considerados para a geração e consulta dos grafos (Fraunhofer AISEC; Joern Project, 2026, 2026). A escolha dependerá da qualidade dos grafos produzidos para Java e C# e do custo de integração. Se o CPG não apresentar ganho suficiente, serão mantidas representações mais simples, como AST, CFG e DFG. SonarQube, CodeQL, Semgrep e analisadores específicos das linguagens poderão fornecer alertas complementares.

5.1.3 MODELOS DE CLASSIFICAÇÃO

O primeiro modelo deverá ser um classificador de referência com baixo custo computacional, utilizado para validar as características extraídas e as métricas de avaliação. A biblioteca dependerá da linguagem escolhida; em Python, o scikit-learn constitui uma alternativa para classificação e validação cruzada (scikit-learn developers, 2026). Posteriormente, poderão ser avaliados CodeBERT e GraphCodeBERT (Feng *et al.*; Guo *et al.*, 2020, 2021). No ecossistema Python, PyTorch e Transformers são alternativas para executar esses modelos (Hugging Face, 2026). Eles não serão requisitos para o produto mínimo viável devido ao maior custo computacional.

5.1.4 RECURSOS DE *HARDWARE*

O projeto não prevê o desenvolvimento de *hardware* específico. O desenvolvimento e os testes iniciais serão realizados nos computadores pessoais da equipe, utilizando CPU e memória convencionais. Caso o treinamento ou a avaliação de modelos mais complexos exija maior capacidade, poderão ser utilizadas GPUs locais ou infraestrutura em nuvem. A necessidade será definida conforme o modelo e o volume de dados selecionados.

5.1.5 AMBIENTE DE DESENVOLVIMENTO E EXECUÇÃO

O código e as configurações serão mantidos em um sistema de controle de versão, o Git. As dependências serão registradas e, quando aplicável, Docker será utilizado para isolar analisadores externos e padronizar a execução.

5.2 TECNOLOGIAS ALTERNATIVAS AVALIADAS

As alternativas serão comparadas por meio de provas de conceito com um conjunto reduzido de arquivos Java e C#. Serão considerados o suporte às linguagens, o tempo de execução, a integração com os demais módulos e a qualidade das características produzidas. Tecnologias mais complexas, como CPGs e modelos pré-treinados, somente serão adotadas se apresentarem ganho mensurável sobre a linha de base. Caso contrário, será mantida a alternativa mais simples para preservar a viabilidade do cronograma e a entrega do protótipo.

6 PROCEDIMENTOS DE TESTE E VALIDAÇÃO DO PROJETO

6.1 PROCEDIMENTOS DE TESTE

A validação da solução será conduzida de forma modular e experimental. Inicialmente, cada módulo do *pipeline* será verificado funcionalmente:

- **Ingestão e normalização:** verificação do processamento de arquivos válidos e tratamento de entradas inconsistentes, garantindo o isolamento da unidade fundamental de análise, que é o **método ou função** modificada no *commit* ou *pull request*;
- **Análise estrutural e extração:** verificação da geração das representações sintáticas básicas (como nós da AST e relações de chamadas) a partir de exemplos controlados em Java (e C# na etapa estendida);
- **Preparação e partição dos dados:** conferência da integridade dos rótulos e garantia de que instâncias de um mesmo repositório não sejam compartilhadas entre os conjuntos de treinamento e teste, mitigando o risco de vazamento de dados (Ding *et al.*, 2024).

A avaliação do módulo de classificação será realizada em um conjunto de teste independente, registrando-se a matriz de confusão e as métricas convencionais de desempenho: precisão, revocação e medida F1.

Para contextualizar os resultados obtidos, a solução será comparada a uma **linha de base (baseline) de referência**, constituída pela execução de regras padrão de detecção para as quatro CWEs em uma ferramenta SAST aberta consagrada (como o Semgrep). O objetivo dessa comparação experimental é avaliar se a adição do classificador de *Machine Learning* sobre as características extraídas proporciona uma tendência de redução de falsos positivos frente aos alertas brutos da análise estática, buscando manter um equilíbrio adequado com a taxa de revocação para não descartar falhas de segurança reais.

Por fim, a integração entre os módulos será testada em fluxo contínuo, assegurando que o código fornecido como entrada percorra as etapas de processamento e resulte na emissão estruturada do relatório de alertas ao usuário.

6.2 CRITÉRIOS DE ACEITE E DESEMPENHO

Os critérios de aceite da solução são apresentados na Tabela 3.

Tabela 3 – Critérios de aceite da solução.

Aspecto	Critério de aceite
Entrada e normalização	Processar arquivos de código em Java (foco do MVP) e C#, isolando as funções modificadas e emitindo mensagens claras em caso de erros de entrada.
Extração de características	Gerar as estruturas sintáticas esperadas e extrair atributos representativos para as funções analisadas, preservando o mapeamento de arquivo e linha.
Gestão de dados	Separar os conjuntos de treino e teste por repositório ou projeto, assegurando avaliação não enviesada.
Classificação e base-line	Apresentar as métricas de precisão, revocação e F1 no conjunto de teste. O desempenho será comparado à linha de base SAST, avaliando o potencial de redução de falsos positivos e a capacidade de retenção de vulnerabilidades reais.
Relatório de saída	Emitir os alertas identificando a CWE associada, o arquivo, a linha aproximada e o nível de confiança da predição.
Integração	Executar o fluxo completo do sistema, desde a leitura do código-fonte até a geração do relatório, sem falhas de comunicação entre os blocos.
Reprodutibilidade	Garantir que o ambiente de execução e os experimentos possam ser reproduzidos com os mesmos conjuntos de dados e configurações.

7 ANÁLISE DOS RISCOS

7.1 ANÁLISE DOS RISCOS

Os riscos foram avaliados qualitativamente quanto à probabilidade de ocorrência e ao impacto sobre o desenvolvimento do projeto. A probabilidade foi classificada como baixa, média ou alta, considerando o escopo proposto, os recursos disponíveis e o cronograma acadêmico. O impacto utiliza a mesma escala e representa o efeito potencial do risco sobre os prazos, a qualidade da solução e a validade dos resultados.

Entre os riscos técnicos, a qualidade dos conjuntos de dados merece atenção, pois bases de vulnerabilidades podem apresentar ruído nos rótulos e diferenças entre os dados utilizados no treinamento e os encontrados em cenários reais. Estudos com o DiverseVul e o PrimeVul mostram que essas limitações podem influenciar diretamente o desempenho e a capacidade de generalização dos modelos (Chen *et al.*; Ding *et al.*; Bhusal *et al.*, 2023, 2024, 2025a).

A geração dos *Code Property Graphs* (CPGs) também constitui um risco relevante. Embora essa representação permita reunir informações de sintaxe, fluxo de controle e fluxo de dados, sua construção depende de ferramentas adicionais e pode elevar a complexidade e o custo computacional do *pipeline* (Yamaguchi, Golde, Arp e Rieck; Chakraborty, Krishna, Ding e Ray, 2014, 2022). Além disso, a integração de LLMs pode introduzir dificuldades de custo, reprodutibilidade e confiabilidade das respostas, especialmente quando o modelo é aplicado a projetos diferentes daqueles utilizados em sua avaliação (Alqarni *et al.*; Ding *et al.*, 2026, 2024).

A Tabela 4 apresenta os principais riscos identificados para o projeto, classificados quanto à sua probabilidade de ocorrência e ao seu impacto sobre o cronograma e os resultados, acompanhados das respectivas estratégias de mitigação.

Tabela 4 – Riscos do projeto, probabilidade, impacto e mitigação.

Risco	Probab.	Impacto	Mitigação
Escassez ou desbalanceamento de dados rotulados de vulnerabilidades	Média	Alto	Utilizar conjuntos de dados públicos já consolidados (por exemplo, PrimeVul e DiverseVul) e técnicas de balanceamento de classes; complementar com dados sintéticos, se necessário
Baixo desempenho do modelo de <i>Machine Learning</i>	Média	Alto	Aplicar validação cruzada, separação clara entre treino e teste e ajuste incremental dos hiperparâmetros dos modelos
Complexidade na integração de <i>Large Language Models</i> ao <i>pipeline</i> de análise	Média	Médio	Tratar o uso de LLMs como módulo complementar e opcional, priorizando inicialmente um classificador de ML mais simples e funcional
Complexidade na geração e integração dos <i>Code Property Graphs</i>	Média	Alto	Restringir inicialmente o escopo a uma linguagem e a um conjunto de CWEs, validar os grafos gerados, fixar as versões das ferramentas de extração e manter representações mais simples como alternativa
Restrição de tempo do cronograma acadêmico	Alta	Alto	Priorizar entregas incrementais, com um MVP funcional antes de adicionar funcionalidades complementares
Limitações de poder computacional para treinamento de modelos	Baixa	Médio	Utilizar infraestrutura em nuvem gratuita ou educacional (Google Colab, Kaggle) sempre que necessário
Falsos positivos ou negativos elevados prejudicando a confiança na ferramenta	Média	Alto	Comparar continuamente com ferramentas de análise estática tradicionais e ajustar os limiares de classificação com base nas métricas obtidas

Os riscos de impacto alto serão tratados com prioridade e revistos ao final das principais etapas do cronograma. Caso um desses riscos se concretize, o escopo dos módulos complementares poderá ser reduzido para preservar a entrega de um protótipo funcional. Em particular, a integração com CPGs e LLMs será conduzida de forma incremental, de modo que o funcionamento básico do classificador e a avaliação das métricas não dependam exclusivamente desses componentes.

8 CRONOGRAMA DO PROJETO

Na Etapa 1, é elaborado e entregue o Formulário de Proposta e, em seguida, a Proposta de Projeto. Na Etapa 2, o presente Plano de Projeto é finalizado tendo o levantamento do estado da arte, a definição da arquitetura, das tecnologias, dos testes e dos riscos, e a redação e revisão do documento. Ele é defendido na Etapa 3. Na Etapa 4 (Elaboração do Projeto Físico), os módulos do sistema são detalhados tecnicamente em profundidade, especificando-se interfaces, modelagem de dados e testes de integração antes da entrega do documento de Projeto Físico, cuja defesa ocorre na Etapa 5. A partir da Etapa 6, já no 1º semestre de 2027 (etapa de *previsão*), o projeto evolui para a construção do protótipo funcional, onde o *pipeline* é implementado, os modelos de *Machine Learning* são treinados e avaliados contra as ferramentas tradicionais de análise estática e os LLMs são integrados experimentalmente à análise contextual, até a entrega do Protótipo e sua defesa na Etapa 7. Na Etapa 8, a implementação é finalizada com ajustes de desempenho e consolidação das métricas comparativas, elaborando-se o Relatório Final e o vídeo de apresentação. Por fim, a Etapa 9 encerra o ciclo com a Defesa Final da implementação do projeto.

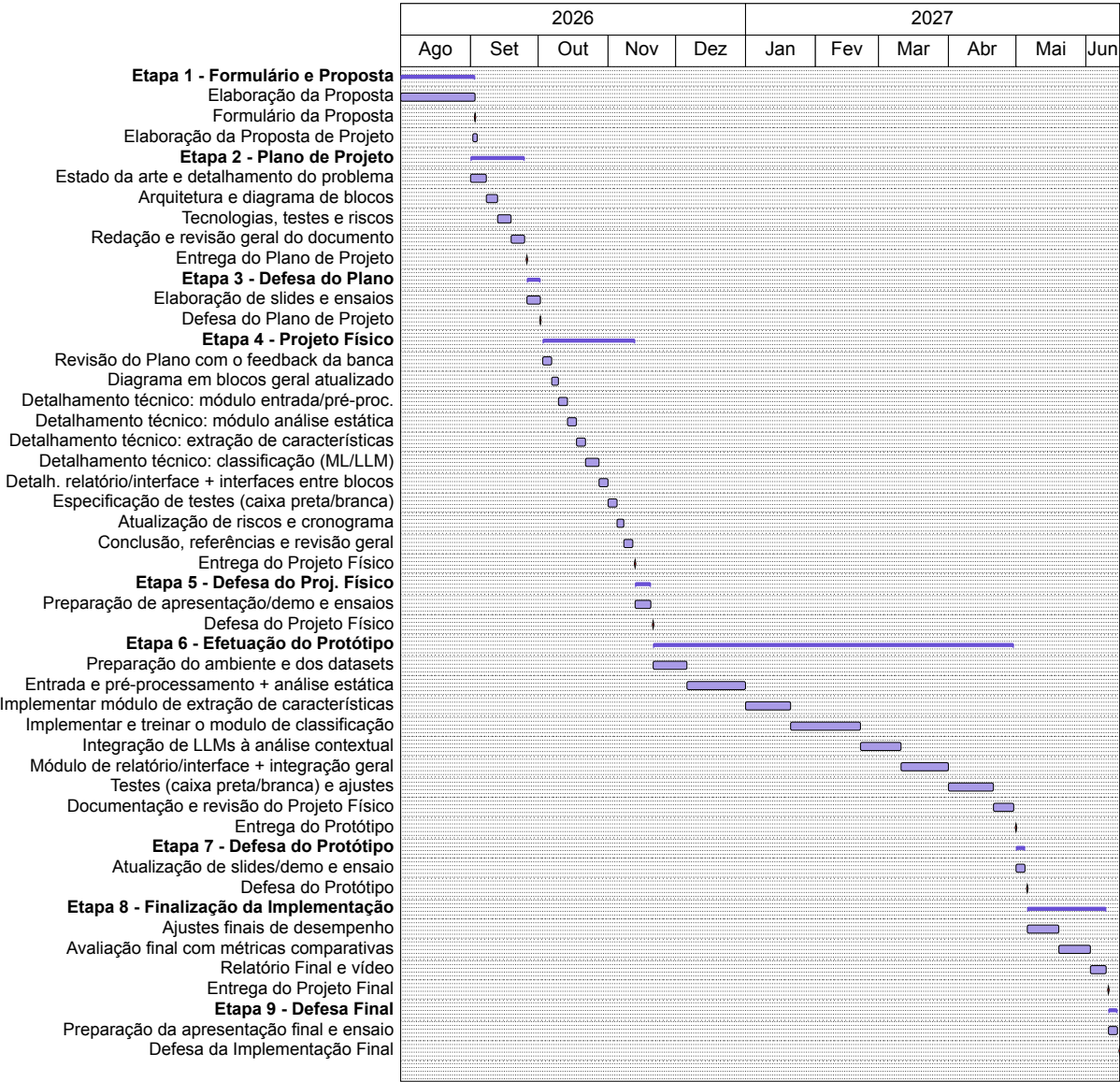


Figura 2 – Cronograma do projeto – Gráfico de Gantt.

9 CONCLUSÃO

Este documento apresentou o Plano de Projeto referente ao desenvolvimento de uma ferramenta de apoio à segurança de software voltada à detecção de vulnerabilidades em código-fonte, com ênfase no cenário de código produzido com o auxílio de ferramentas de Inteligência Artificial. Como discutido no Capítulo 2, estudos como os de Pearce et al. (Pearce *et al.*, 2022), Perry et al. (Perry, Srivastava, Kumar e Boneh, 2023) e o relatório da Veracode (Veracode, 2025) evidenciam que o código gerado com o apoio de assistentes de IA apresenta índices de vulnerabilidade significativamente superiores aos do código escrito exclusivamente por humanos, com padrões inseguros que se repetem entre diferentes projetos e modelos (Wu *et al.*; State of Cyber, 2025, 2026). Esse cenário reforça a relevância prática do problema tratado neste trabalho.

A revisão do estado da arte, apresentada no Capítulo 3, mostrou que as ferramentas tradicionais de análise estática (SAST), apesar de sua maturidade e explicabilidade, apresentam limitações bem documentadas relacionadas à elevada taxa de falsos positivos e à dificuldade de compreender o contexto semântico do código (TechScience; Yang *et al.*, 2025, 2025). Modelos de *Machine Learning* baseados em grafos, como o Devign (Zhou *et al.*, 2019), e modelos pré-treinados para código, como CodeBERT (Feng *et al.*, 2020) e GraphCodeBERT (Guo *et al.*, 2021), representam avanços relevantes na captura de relações estruturais e semânticas do código-fonte, mas ainda enfrentam desafios de generalização e de custo computacional (Chakraborty, Krishna, Ding e Ray; Ding *et al.*, 2022, 2024). Nesse contexto, abordagens híbridas que combinam *Code Property Graphs* (Yamaguchi, Golde, Arp e Rieck, 2014) com *Large Language Models*, como o LLMxCPPG (Kouliaridis *et al.*, 2025), o SV-GEN (Zhou *et al.*, 2026) e o ZeroFalse (Yang *et al.*, 2025), demonstraram potencial para reduzir de forma expressiva os falsos alarmes gerados por analisadores estáticos, o que motivou a proposta de arquitetura apresentada neste projeto.

A partir dessa fundamentação, o Capítulo 4 definiu uma arquitetura modular composta por módulos de ingestão e normalização, análise sintática e estrutural,

representação intermediária e extração de características, detecção e classificação de vulnerabilidades, explicação e priorização de alertas, e geração de relatórios. O escopo do MVP concentrará o *pipeline* básico de análise estática e classificação na linguagem prioritária Java, adotando C# como extensão experimental de portabilidade, com foco nas categorias CWE-22, CWE-78, CWE-79 e CWE-89, refletindo o mesmo recorte de vulnerabilidades investigado por trabalhos correlatos de medição de segurança de código gerado por IA (Pearce *et al.*; Wu *et al.*, 2022, 2025).

No que diz respeito às tecnologias, discutidas no Capítulo 5, a decisão sobre o uso de *Code Property Graphs* dependerá de provas de conceito com ferramentas como Joern e CPG/Codyze (Yamaguchi, Golde, Arp e Rieck, 2014), enquanto a etapa de classificação poderá evoluir de modelos de referência mais simples para arquiteturas baseadas em *Transformers*, como CodeBERT e GraphCodeBERT (Feng *et al.*; Guo *et al.*, 2020, 2021). Os procedimentos de teste e validação definidos no Capítulo 6 adotam a separação dos dados por projeto ou repositório como salvaguarda metodológica, prática recomendada por Ding *et al.* (Ding *et al.*, 2024) para evitar a superestimação de desempenho observada em benchmarks tradicionais. Da mesma forma, os riscos identificados no Capítulo 7 – como a qualidade dos dados rotulados e a dificuldade de generalização entre distribuições distintas de projetos (Chen *et al.*; Ding *et al.*; Bhusal *et al.*, 2023, 2024, 2025a) – orientaram diretamente as estratégias de mitigação propostas, entre elas o uso de bases consolidadas como DiverseVul e PrimeVul e a adoção de uma abordagem incremental para a integração de CPGs e LLMs ao *pipeline*.

O cronograma apresentado no Capítulo 8 organiza essas atividades ao longo das etapas previstas para o Projeto Final de Curso, desde a elaboração deste Plano de Projeto até a defesa da implementação final, prevista para o primeiro semestre de 2027. Dessa forma, o presente trabalho posiciona-se como uma contribuição à etapa intermediária do ciclo de desenvolvimento seguro de software – anterior ao *merge* e à integração definitiva do código à *branch* principal –, complementando as abordagens de medição de insegurança e de indução de geração mais segura discutidas na literatura (Liu *et al.*; Wu *et al.*, 2025, 2025). Espera-se que, ao final do desenvolvimento, a ferramenta proposta contribua para a redução de vulnerabilidades introduzidas no fluxo contínuo de desenvolvimento (incluindo código assistido por IA), oferecendo aos desenvolvedores alertas mais precisos, contextualizados e priorizados,

e reduzindo a dependência de revisões manuais exaustivas para a identificação de falhas de segurança.

REFERÊNCIAS

ALQARNI, M. *et al.* A survey on large language models in software security: Opportunities and threats. **Computers**, MDPI, v. 15, n. 4, 2026. Disponível em: <https://www.mdpi.com/2073-431X/15/4/226>. Acesso em: 15 set. 2026.

BHUSAL, D. *et al.* **Evaluating the Out-of-Distribution Generalization of Vulnerability Detection Models**. 2025. ArXiv:2507.21817. Disponível em: <https://arxiv.org/abs/2507.21817>. Acesso em: 15 set. 2026.

BHUSAL, D. *et al.* **Out of Distribution, Out of Luck: How Well Can LLMs Trained on Vulnerability Datasets Detect Top 25 CWE Weaknesses?** 2025. ArXiv:2507.21817. Disponível em: <https://arxiv.org/abs/2507.21817>. Acesso em: 15 set. 2026.

CHAKRABORTY, S.; KRISHNA, R.; DING, Y.; RAY, B. Deep learning based vulnerability detection: Are we there yet? **IEEE Transactions on Software Engineering**, v. 48, n. 9, p. 3280–3296, 2022.

CHEN, Y.; DING, Z.; ALOWAIN, L.; CHEN, X.; WAGNER, D. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In: **Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)**. [S.l.: s.n.], 2023. p. 654–668.

DING, Y.; FU, Y.; IBRAHIM, O.; SITAWARIN, C.; CHEN, X.; ALOMAIR, B.; WAGNER, D.; RAY, B.; CHEN, Y. **Vulnerability Detection with Code Language Models: How Far Are We?** 2024. ArXiv:2403.18624. Disponível em: <https://arxiv.org/abs/2403.18624>. Acesso em: 15 set. 2026.

ELDER, S.; RAHMAN, N. Z.; FU, G.; WILLIAMS, L. A survey on software vulnerability detection methods: Sast, dast, iast and manual penetration testing. 2023. Disponível em: <https://arxiv.org/abs/2312.17726>. Acesso em: 17 set. 2026.

FAN, J.; LI, Y.; WANG, S.; NGUYEN, T. N. A c/c++ code vulnerability dataset with code changes and cve summaries. In: **Proceedings of the 17th International Conference on Mining Software Repositories (MSR)**. [S.l.: s.n.], 2020. p. 508–512.

FENG, Z.; GUO, D.; TANG, D.; DUAN, N.; FENG, X.; GONG, M.; SHOU, L.; QIN, B.; LIU, T.; JIANG, D.; ZHOU, M. Codebert: A pre-trained model for programming and natural languages. In: **Findings of the Association for Computational Linguistics: EMNLP 2020**. [S.l.: s.n.], 2020. p. 1536–1547.

Fraunhofer AISEC. **Code Property Graph: A Language-Independent Analysis Platform for Source Code**. 2026. Documentação técnica. Disponível em: <https://www.codyze.io/>. Acesso em: 23 set. 2026.

GUO, D.; REN, S.; LU, S.; FENG, Z.; TANG, D.; LIU, S.; ZHOU, L.; DUAN, N.; SVYATKOVSKIY, A.; FU, S.; TUFANO, M.; DENG, S. K.; CLEMENT, C.; DRAIN, D.; SUNDARESAN, N.; YIN, J.; JIANG, D.; ZHOU, M. Graphcodebert: Pre-training code representations with data flow. In: **International Conference on Learning Representations (ICLR)**. [S.l.: s.n.], 2021.

HASAN, M. B. U. *et al.* **SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection**. 2025. ArXiv:2505.19828. Disponível em: <https://arxiv.org/abs/2505.19828>. Acesso em: 15 set. 2026.

Hugging Face. **Transformers Documentation**. 2026. Documentação técnica. Disponível em: <https://huggingface.co/docs/transformers/index>. Acesso em: 23 set. 2026.

Joern Project. **Joern Documentation**. 2026. Documentação técnica. Disponível em: <https://docs.joern.io/>. Acesso em: 23 set. 2026.

KOULIARIDIS, V. *et al.* **LLMxCPG: Context-Aware Vulnerability Detection Through Large Language Models and Code Property Graphs**. 2025. ArXiv preprint.

LI, Y. *et al.* Machine learning for source code vulnerability detection. **IEEE Security & Privacy**, v. 20, n. 5, p. 39–48, 2022.

LIU, Y. *et al.* **An Empirical Study on LLM-Driven Secure Code Generation**. 2025. ArXiv:2506.23034. Disponível em: <https://arxiv.org/abs/2506.23034>. Acesso em: 15 set. 2026.

OWASP Foundation. **OWASP DevSecOps Guideline: Application Security Testing**. 2026. Documento técnico. Disponível em: <https://owasp.org/www-project-devsecops-guideline/>. Acesso em: 17 set. 2026.

PEARCE, H.; AHMAD, B.; TAN, B.; DOLAN-GAVITT, B.; KARRI, R. Asleep at the keyboard? assessing the security of github copilot's code contributions. In: **2022 IEEE Symposium on Security and Privacy (SP)**. [S.l.: s.n.], 2022. p. 754–768.

PERRY, N.; SRIVASTAVA, M.; KUMAR, D.; BONEH, D. Do users write more insecure code with ai assistants? In: **Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)**. [S.l.: s.n.], 2023. p. 2785–2799.

RANASINGHE, A. *et al.* Comparative evaluation of approaches and tools for effective vulnerability detection. **PMC**, 2025. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12190248/>. Acesso em: 17 set. 2026.

Safeguard.sh. **How Modern SAST Engines Track Data Flow**. 2024. Blog técnico. Disponível em: <https://safeguard.sh/resources/blog/modern-sast-how-static-analysis-works>. Acesso em: 15 set. 2026.

scikit-learn developers. **scikit-learn: Machine Learning in Python**. 2026. Documentação técnica. Disponível em: <https://scikit-learn.org/stable/>. Acesso em: 23 set. 2026.

State of Cyber. **AI-Assisted Code Introduces 4.4x More Security Vulnerabilities Than Human-Written Code, New Research Finds**. 2026. The Tennessean. Disponível em: <https://www.tennessean.com/press-release/story/223693/ai-assisted-code-introduces-4-4x-more-security-vulnerabilities-than-human-written-code-new-research-finds/>. Acesso em: 15 set. 2026.

TechScience. Static analysis techniques for secure software: A systematic review. **Journal of Cyber Security**, v. 7, n. 1, 2025. Disponível em: <https://www.techscience.com/JCS/v7n1/64045/html>. Acesso em: 15 set. 2026.

Veracode. **2025 GenAI Code Security Report**. 2025. Relatório técnico. Disponível em: <https://www.veracode.com/resources/reports/2025-genai-code-security-report/>. Acesso em: 15 set. 2026.

WANG, X. *et al.* **AdaTaint: LLM-Driven Adaptive Source-Sink Identification and Neuro-Symbolic Taint Analysis**. 2025. ArXiv:2511.04023. Disponível em: <https://arxiv.org/abs/2511.04023>. Acesso em: 15 set. 2026.

WANG, Y. *et al.* A generative ai cybersecurity risks mitigation model for code generation: Using ann-ism hybrid approach. **Scientific Reports**, 2025. Disponível em: <https://www.nature.com/articles/s41598-025-34350-3>. Acesso em: 15 set. 2026.

WEN, C. *et al.* **Reducing False Positives in Static Bug Detection via LLM-Enhanced Path Feasibility Analysis**. 2025. ArXiv:2506.10322. Disponível em: <https://arxiv.org/abs/2506.10322>. Acesso em: 15 set. 2026.

WU, Z. *et al.* **AI Code in the Wild: Measuring Security Risks and Mitigations**. 2025. ArXiv:2512.18567. Disponível em: <https://arxiv.org/abs/2512.18567>. Acesso em: 15 set. 2026.

YAMAGUCHI, F.; GOLDE, N.; ARP, D.; RIECK, K. Modeling and discovering vulnerabilities with code property graphs. In: **2014 IEEE Symposium on Security and Privacy**. [S.l.: s.n.], 2014. p. 590–604.

YANG, X. *et al.* **ZeroFalse: Improving Precision in Static Analysis with LLMs**. 2025. ArXiv:2510.02534. Disponível em: <https://arxiv.org/abs/2510.02534>. Acesso em: 15 set. 2026.

ZHOU, X. *et al.* Sv-gen: Synergizing llm-empowered variable semantics and graph transformers for vulnerability detection. **Future Internet**, MDPI, v. 18, n. 5, 2026.

ZHOU, Y.; LIU, S.; SLOW, J.; DU, X.; LIU, Y. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: **Advances in Neural Information Processing Systems**. [S.l.: s.n.], 2019. v. 32.